Yasmani Viñamagua

VERSIÓN 1.0
OCTUBRE 2026

MANUAL DE ESTUDIO

Git y GitHub

Guía de estudio y consulta sobre control de versiones, ramas y colaboración.

AUTOR
Yasmani Viñamagua

Índice

01 Introducción **5**

1.1 Convenciones del documento 5

02 Fundamentos del control de versiones **6**

2.1 Conceptos esenciales 6

2.2 Estados y áreas de trabajo 6

2.3 Síntesis 7

03 Configuración inicial **8**

3.1 Identidad y editor 8

3.2 Finales de línea y consulta de la configuración 8

3.3 Síntesis 9

04 Flujo básico de trabajo **10**

4.1 Creación del primer repositorio 10

4.2 Registro de cambios 10

4.3 Eliminación de archivos 10

4.4 Revisión del contenido y del historial 10

4.5 Exclusión de archivos con .gitignore 11

4.6 Síntesis 11

05 Inspección del historial **12**

5.1 Comparación de cambios 12

5.2 Historial y autoría 12

5.3 Búsqueda en archivos y en el historial 12

5.4 Síntesis 12

06 Deshacer cambios y recuperar versiones **13**

6.1 Descartar cambios en archivos 13

6.2 Reset y revert 13

6.3 Recuperación con reflog 13

6.4 Limpieza de archivos sin seguimiento 14

6.5 Síntesis 14

07 Ramas y fusión 15

7.1 Gestión de ramas 15

7.2 Resolución de conflictos de fusión 15

7.3 Reorganización y selección de commits 16

7.4 Trabajo a medias: stash y worktree 16

7.5 Síntesis 16

08 Repositorios remotos y GitHub 17

8.1 Conexión y sincronización 17

8.2 Autenticación con claves SSH 17

8.3 Publicación con GitHub Pages 18

8.4 Síntesis 18

09 Colaboración en GitHub 19

9.1 Flujo de trabajo con ramas de funcionalidad 19

9.2 Contribución a proyectos ajenos 19

9.3 Referencias entre issues y commits 19

9.4 Síntesis 20

10 Etiquetas (tags) 21

10.1 Uso de etiquetas 21

10.2 Síntesis 21

11 Buenas prácticas y mensajes de commit 22

11.1 Convención de mensajes 22

11.2 Recomendaciones de uso 23

11.3 Síntesis 23

12 Productividad: alias y GitHub CLI 24

12.1 Alias 24

12.2 GitHub CLI 24

12.3 Atajos de teclado en github.com 24

12.4 Síntesis 25

13 Markdown para documentación 26

13.1 Sintaxis básica 26

13.2 Bloques de código, tablas y avisos 26

13.3 Síntesis 26

Glosario 27

Referencias 28

01 Introducción

1.1 Convenciones del documento

- Los procedimientos y comandos proceden de apuntes de curso (Vega, 2021) y de una hoja de referencia de uso frecuente (DevTalles, s. f.), ordenados según su dependencia conceptual y complementados con información verificada en fuentes oficiales.
- Los comandos y fragmentos de código se presentan en bloques con fuente monoespaciada. Los términos entre signos `< >` son marcadores que deben sustituirse por valores reales (por ejemplo, `<rama>`).
- Los procedimientos se redactan en infinitivo o en forma impersonal con «se».
- Los recuadros identifican información de apoyo: **Nota** (aclaración), **Importante** (advertencia), **Consejo** (recomendación práctica) y **Para ampliar** (información complementaria verificada, ajena a los apuntes originales).
- El nombre de la rama principal varía según el repositorio (`main` o `master`). Los ejemplos emplean `main`; debe sustituirse por el nombre vigente en cada proyecto.

> **NOTA**
>
> Los ejemplos se ejecutan en una terminal Bash (Git Bash en Windows). Los comandos son equivalentes en otras terminales, salvo indicación contraria.

02 Fundamentos del control de versiones

2.1 Conceptos esenciales

Un **sistema de control de versiones** registra los cambios realizados sobre un conjunto de archivos a lo largo del tiempo, de modo que sea posible consultar versiones anteriores, comparar cambios y colaborar sin sobrescribir el trabajo de otras personas. **Git** es un sistema de control de versiones distribuido: cada copia del proyecto contiene el historial completo. **GitHub** es un servicio en línea que aloja repositorios Git y añade herramientas de colaboración, como las solicitudes de extracción y los *issues*.

Tabla 1
Vocabulario Esencial de Git y GitHub

Término	Definición
Repositorio	Proyecto completo con todo su historial de cambios.
Commit	Instantánea guardada de los cambios en un momento dado.
Rama (*branch*)	Línea de trabajo paralela e independiente.
Remoto	Copia del repositorio alojada en un servidor, por ejemplo GitHub.
Origin	Nombre por defecto del remoto principal.
HEAD	Apuntador al commit o rama sobre el que se trabaja actualmente.
Stage	Área de preparación de los cambios que formarán parte del próximo commit.
Pull request	Propuesta para fusionar una rama, sometida a revisión.
Fork	Copia de un repositorio ajeno en la propia cuenta.

Nota. Adaptado de DevTalles (s. f.).

2.2 Estados y áreas de trabajo

Un archivo versionado puede hallarse en uno de tres estados: *modificado* (cambiado, pero no registrado), *preparado* (marcado para formar parte del próximo commit) o *confirmado* (almacenado de forma segura en la base de datos local). A estos estados corresponden tres áreas: el directorio de trabajo, el área de preparación (*staging area* o índice) y el directorio de Git (Chacon y Straub, 2014).

Figura 1

Flujo entre las Tres Áreas de Git



Nota. Elaboración propia a partir de Chacon y Straub (2014).

> PARA AMPLIAR
>
> El área de preparación es un archivo, generalmente ubicado en el directorio de Git, que almacena la información de lo que formará parte del próximo commit; su nombre técnico es «índice» (Chacon y Straub, 2014).

2.3 Síntesis

- Git registra el historial del proyecto como una secuencia de commits.
- Los cambios fluyen del directorio de trabajo al área de preparación y de allí al repositorio.
- GitHub aloja repositorios y facilita la colaboración, pero Git funciona sin él.

03 Configuración inicial

3.1 Identidad y editor

La configuración inicial se realiza una sola vez por equipo con el modificador `--global`. El nombre y el correo quedan asociados a todos los commits.

```
git config --global user.name "Nombre Apellido"
git config --global user.email "correo@dominio.com"
git config --global core.editor "code --wait"
git config --global init.defaultBranch main
```

La última instrucción establece `main` como rama predeterminada de los repositorios nuevos, y la tercera configura Visual Studio Code como editor.

> **IMPORTANTE**
>
> En los comandos deben emplearse comillas rectas ("), no tipográficas (“ ”). Las comillas tipográficas, frecuentes al copiar texto desde procesadores de texto o notas, provocan errores o valores incorrectos.

3.2 Finales de línea y consulta de la configuración

Windows y macOS/Linux utilizan finales de línea distintos. El valor `core.autocrlf` permite normalizarlos: `true` en Windows e `input` en macOS y Linux.

```
git config --global core.autocrlf true      # Windows
git config --global core.autocrlf input     # macOS / Linux
```

Tabla 2
Comandos de Consulta de la Configuración

Comando	Función
`git config --list`	Muestra la configuración vigente.
`git config --list --show-origin`	Indica en qué archivo está definido cada valor.
`git config --global -e`	Abre el archivo de configuración global para editarlo.
`git config -h`	Lista las opciones disponibles.

Nota. Adaptado de Vega (2021) y DevTalles (s. f.).

3.3 Síntesis

- La identidad (`user.name` y `user.email`) se configura una vez por equipo.
- `--show-origin` permite localizar el archivo en que está definido cada valor.
- El nombre de la rama predeterminada puede fijarse con `init.defaultBranch` .

04 Flujo básico de trabajo

4.1 Creación del primer repositorio

El procedimiento parte de una carpeta de proyecto vacía:

1. Abrir la terminal y crear la carpeta del proyecto (`mkdir proyecto1`).
2. Ingresar a la carpeta (`cd proyecto1`) y comprobar la ubicación con `pwd` .
3. Inicializar el repositorio con `git init` .
4. Verificar que se creó la carpeta oculta `.git` con `ls -al` .
5. Crear un archivo de texto y guardarlo en la carpeta.

4.2 Registro de cambios

El ciclo básico consiste en comprobar el estado, preparar los cambios y confirmarlos con un mensaje descriptivo.

```
git status                      # estado de los archivos
git add nombre_archivo.txt      # prepara un archivo
git add .                       # prepara todos los cambios
git commit -m "Mensaje"         # confirma lo preparado
git commit -am "Mensaje"        # prepara y confirma archivos ya rastreados
```

> CONSEJO
>
> Si se ejecuta `git commit` sin el modificador `-m` , Git abre el editor configurado para redactar el mensaje. Si el editor es Vim, se guarda y se sale con `Esc` seguido de `:wq` (o `Shift + Z Z`).

4.3 Eliminación de archivos

El comando `git rm nombre_archivo.txt` elimina el archivo del directorio de trabajo y lo retira del seguimiento. Con `git rm --cached nombre_archivo.txt` el archivo deja de rastrearse, pero se conserva en el disco, lo que resulta útil cuando se añadió por error un archivo que debía ignorarse.

4.4 Revisión del contenido y del historial

```
cat nombre_archivo.txt          # muestra el contenido
code nombre_archivo.txt         # abre el archivo en el editor
git log nombre_archivo.txt      # historial del archivo
git log --oneline               # historial compacto
git log --stat                  # archivos modificados en cada commit
```

4.5 Exclusión de archivos con .gitignore

El archivo `.gitignore` enumera los patrones de archivos que Git no debe rastrear. Se crea en la raíz del proyecto y se versiona como cualquier otro archivo.

```
node_modules/
*.log
dist/
.env
```

Cada línea excluye, respectivamente, una carpeta completa, todos los archivos con una extensión, las carpetas de compilación y las variables de entorno.

> **IMPORTANTE**
>
> Los secretos (contraseñas, claves de API, archivos `.env`) no deben subirse nunca a un repositorio. Si se añadió un archivo antes de incluirlo en `.gitignore`, debe retirarse del seguimiento con `git rm --cached` (DevTalles, s. f.).

4.6 Síntesis

- El ciclo básico es `status` → `add` → `commit`.
- `git rm --cached` detiene el seguimiento sin borrar el archivo.
- `.gitignore` evita versionar archivos generados o confidenciales.

05 Inspección del historial

5.1 Comparación de cambios

```
git diff                          # cambios aún sin preparar
git diff --staged                 # cambios ya preparados
git diff rama1..rama2             # compara dos ramas
git diff <commit_reciente> <commit_antiguo>   # compara dos commits
git show <commit>                 # cambios introducidos por un commit
git show <commit>:archivo         # archivo tal como estaba en ese commit
```

5.2 Historial y autoría

```
git log --oneline --graph --all  # historial gráfico de todas las ramas
git log -p                        # historial con el diff de cada commit
git log --author="nombre"         # commits de una persona
git log -- <archivo>              # commits que tocaron un archivo
git shortlog -sn                  # número de commits por persona
git blame <archivo>               # autor y commit de cada línea
git blame <archivo> -L35,50       # solo las líneas 35 a 50
```

La opción `git shortlog -sn --all` incluye las ramas no fusionadas, y `--no-merges` excluye los commits de fusión.

5.3 Búsqueda en archivos y en el historial

Tabla 3

Comandos de Búsqueda

Comando	Ámbito	Función
`git grep palabra`	Archivos	Localiza los archivos que contienen la palabra.
`git grep -n palabra`	Archivos	Añade el número de línea.
`git grep -c palabra`	Archivos	Cuenta las coincidencias por archivo.
`git log --grep="texto"`	Commits	Busca en los mensajes de commit.
`git log -S "palabra"`	Commits	Encuentra los commits que añadieron o quitaron esa cadena.
`git bisect start`	Commits	Búsqueda binaria del commit que introdujo un error.

Nota. `git grep` examina archivos; `git log`, el historial. Adaptado de Vega (2021) y DevTalles (s. f.).

5.4 Síntesis

- `git diff` compara estados; `git show` inspecciona un commit.
- `git blame` y `git shortlog` atribuyen cambios a personas.
- `git grep` busca en archivos y `git log` en el historial.

06 Deshacer cambios y recuperar versiones

6.1 Descartar cambios en archivos

```
git restore <archivo>            # vuelve al último commit
git restore --staged <archivo>   # saca el archivo del stage sin perder cambios
git restore .                    # descarta todos los cambios sin guardar
git checkout <commit> <archivo>  # muestra cómo era el archivo en ese commit
git commit --amend -m "nuevo mensaje"   # corrige el último commit
```

> **IMPORTANTE**
>
> `git restore .` descarta de forma irreversible los cambios no confirmados. Conviene revisar `git status` y `git diff` antes de ejecutarlo.

6.2 Reset y revert

El comando `git reset` mueve la rama actual a otro commit; según el modificador, conserva o descarta el trabajo posterior. En cambio, `git revert` crea un commit nuevo que invierte los efectos de otro, por lo que es la opción segura cuando los commits ya se compartieron.

Tabla 4
Variantes de git reset

Comando	Efecto
`git reset --soft HEAD~1`	Deshace el último commit y conserva los cambios en el stage.
`git reset --mixed HEAD~1`	Deshace el commit y deja los cambios sin preparar (comportamiento predeterminado).
`git reset --hard <commit>`	Regresa a ese commit y descarta todo lo posterior, incluido el stage.
`git revert <commit>`	Crea un commit que revierte otro; adecuado para repositorios compartidos.

Nota. Adaptado de Vega (2021) y DevTalles (s. f.).

6.3 Recuperación con reflog

El comando `git reflog` registra los movimientos de HEAD, incluso los commits que ya no son alcanzables desde ninguna rama. Permite volver al estado en que el proyecto funcionaba:

```
git reflog                       # lista el hash y la posición de HEAD
git reset --soft <hash>          # vuelve conservando lo que hay en staging
git reset --hard <hash>          # vuelve descartando todo, incluido el staging
```

6.4 Limpieza de archivos sin seguimiento

```
git clean --dry-run             # muestra qué se borraría
git clean -f                    # borra los archivos listados
git clean -fd                   # borra también carpetas
```

> **IMPORTANTE**
>
> Los archivos eliminados con `git clean` no pueden recuperarse desde Git. Debe ejecutarse siempre primero con `--dry-run` .

6.5 Síntesis

- `restore` descarta cambios en archivos; `reset` mueve la rama; `revert` añade un commit inverso.
- `--hard` y `clean` son destructivos: deben usarse con verificación previa.
- `reflog` permite recuperar commits aparentemente perdidos.

07 Ramas y fusión

7.1 Gestión de ramas

Una rama permite desarrollar una funcionalidad o corregir un error sin alterar la línea principal. La práctica recomendada es una rama por funcionalidad o error, evitando trabajar directamente en la rama principal (DevTalles, s. f.).

```
git branch                      # lista las ramas (* indica la actual)
git switch -c <rama>            # crea una rama y se mueve a ella
git switch <rama>               # cambia de rama
git checkout -b <rama>          # equivalente clásico de switch -c
git merge <rama>                # fusiona la rama indicada en la actual
git branch -d <rama>            # elimina una rama ya fusionada
git branch -D <rama>            # fuerza la eliminación
git branch -m <nuevo-nombre>    # renombra la rama actual
git show-branch --all           # historia de todas las ramas
gitk                            # visualización gráfica
```

> **NOTA**
>
> Para fusionar, el procedimiento consiste en situarse en la rama que recibirá los cambios (`git switch main`) y ejecutar `git merge <rama>` .

7.2 Resolución de conflictos de fusión

Cuando dos ramas modifican la misma línea, Git no puede decidir cuál conservar y marca el conflicto en el archivo:

```
<<<<<<< HEAD
versión de la rama actual
=======
versión de la rama entrante
>>>>>>> otra-rama
```

1. Editar el archivo y dejar el contenido final, eliminando los marcadores.
2. Marcar el conflicto como resuelto con `git add <archivo>` .
3. Cerrar la fusión con `git commit` .
4. Para cancelar el proceso: `git merge --abort` .

Tabla 5
Opciones de Resolución de Conflictos en el Editor

Opción	Efecto
Accept Current Change	Conserva la versión de la rama actual (HEAD) y descarta la entrante.
Accept Incoming Change	Acepta los cambios de la rama que se fusiona.
Accept Both Changes	Une ambas versiones; exige revisar que el resultado sea coherente.
Compare Changes	Abre una comparación visual para decidir línea por línea.

Nota. Opciones del editor Visual Studio Code. Adaptado de Vega (2021).

7.3 Reorganización y selección de commits

```
git rebase main                 # coloca los commits de la rama sobre main
git rebase -i HEAD~3            # rebase interactivo: squash, reword, edit
git cherry-pick <commit>        # trae un commit concreto a la rama actual
```

> **IMPORTANTE**
>
> No debe aplicarse `rebase` a commits ya publicados en un remoto compartido, pues reescribe el historial y afecta a quienes trabajan sobre él (DevTalles, s. f.).

7.4 Trabajo a medias: stash y worktree

El comando `git stash` guarda temporalmente los cambios en curso y deja el directorio limpio, de modo que pueda cambiarse de rama.

```
git stash                       # guarda los cambios en curso
git stash list                  # lista los stashes
git stash pop                   # recupera el último y lo elimina de la lista
git stash apply                 # lo recupera y lo conserva
git stash branch <nueva_rama>   # crea una rama a partir del stash
git stash drop                  # elimina el último stash
```

Como alternativa, `git worktree add ../hotfix <rama>` abre otra rama en una carpeta distinta sin tocar el trabajo actual; se retira con `git worktree remove ../hotfix`.

7.5 Síntesis

- Las ramas aíslan el trabajo; `merge` lo integra.
- Un conflicto se resuelve editando, preparando y confirmando.
- `stash` y `worktree` permiten cambiar de contexto sin perder trabajo.

08 Repositorios remotos y GitHub

8.1 Conexión y sincronización

```
git remote add origin <url>      # conecta el repositorio local con el remoto
git remote -v                    # lista remotos (fetch y push)
git push -u origin main          # sube la rama y la vincula (primera vez)
git push                         # sube los commits
git pull                         # descarga y fusiona cambios del remoto
git pull --rebase                # descarga y reaplica los commits propios encima
git fetch                        # descarga sin fusionar
git clone <url>                  # descarga un repositorio con su historial
git clone --depth 1 <url>        # clon superficial: solo el último commit
```

Si el repositorio remoto y el local se crearon por separado y tienen historiales sin relación, `git pull origin <rama> --allow-unrelated-histories` obliga a fusionarlos. Para consultar las ramas remotas se emplea `git branch -r`, y `git branch -a` muestra las locales y las remotas.

> **IMPORTANTE**
>
> El nombre de la rama debe coincidir en el repositorio local y en GitHub. Los apuntes originales emplean `master`; en repositorios actuales lo habitual es `main`. Conviene verificar el nombre con `git branch` antes de ejecutar `push` o `pull`.

8.2 Autenticación con claves SSH

Las claves SSH permiten comunicarse con GitHub sin introducir credenciales en cada operación. El procedimiento general es el siguiente:

1. Generar el par de claves (pública y privada) con `ssh-keygen`.
2. Iniciar el agente y agregar la clave privada (`eval "$(ssh-agent -s)"` y `ssh-add <ruta_de_la_clave>`).
3. Copiar el contenido de la **clave pública** (archivo `.pub`) y pegarlo en GitHub, en *Settings* → *SSH and GPG keys*.
4. Cambiar la dirección del remoto a la versión SSH: `git remote set-url origin <direccion_SSH>`.
5. Verificar con `git remote -v` y sincronizar con `git pull`.

> **PARA AMPLIAR**
>
> GitHub recomienda generar la clave con el algoritmo Ed25519: `ssh-keygen -t ed25519 -C "correo@dominio.com"`. El comando `ssh-keygen -t rsa -b 4096 -C "correo@dominio.com"`, empleado en los apuntes, queda reservado para sistemas antiguos que no admiten Ed25519 (GitHub, s. f.-b). En ese caso la clave privada se denomina `id_rsa` y la pública `id_rsa.pub`.

Si la conexión por el puerto habitual falla (por ejemplo, por restricciones de red), puede configurarse GitHub sobre el puerto 443 añadiendo al archivo `~/.ssh/config`:

```
Host github.com
  Hostname ssh.github.com
  Port 443
```

8.3 Publicación con GitHub Pages

GitHub Pages publica contenido estático (HTML, CSS y JavaScript) directamente desde un repositorio. El procedimiento es el siguiente:

1. Crear un repositorio en GitHub y clonarlo localmente.
2. Añadir un archivo `index.html`, confirmar los cambios y subirlos.
3. En el repositorio, abrir *Settings* → *Pages*, y en *Build and deployment* seleccionar *Source* → *Deploy from a branch*, elegir la rama y la carpeta (raíz o `/docs`) y guardar.
4. Acceder a la dirección publicada desde el navegador.

> **PARA AMPLIAR**
>
> Además de «Deploy from a branch», GitHub ofrece publicar mediante GitHub Actions, opción indicada cuando se utiliza un proceso de compilación distinto de Jekyll o no se desea una rama dedicada a los archivos compilados (GitHub, s. f.-a). Los apuntes originales mencionan el nombre `nombre_de_usuario.github.io` para el repositorio del sitio personal.

8.4 Síntesis

- `push` envía commits al remoto; `pull` los trae y fusiona; `fetch` solo los descarga.
- Las claves SSH evitan introducir credenciales en cada operación; solo se comparte la clave pública.
- GitHub Pages publica sitios estáticos desde una rama.

09 Colaboración en GitHub

9.1 Flujo de trabajo con ramas de funcionalidad

1. Actualizar la rama principal con `git pull`.
2. Crear la rama: `git switch -c mi-funcionalidad`.
3. Trabajar y confirmar los cambios (`git commit -am "mensaje"`).
4. Subir la rama: `git push -u origin mi-funcionalidad`.
5. En GitHub, abrir un *pull request* con la rama principal como base y la rama creada como comparación.
6. Atender la revisión y las correcciones hasta obtener la aprobación.
7. Si no hay conflictos, fusionar el *pull request* (*Merge pull request*) y eliminar la rama.
8. Actualizar localmente: `git switch main` y `git pull`.

9.2 Contribución a proyectos ajenos

1. Realizar un *fork*, que copia el repositorio a la cuenta propia.
2. Clonar el *fork*: `git clone <url-de-tu-fork>`.
3. Crear una rama, realizar los cambios y subirlos al *fork*.
4. Abrir un *pull request* hacia el repositorio original.

Para mantener el *fork* actualizado se conecta el repositorio original como remoto *upstream*:

```
git remote add upstream <url-original>
git fetch upstream
git merge upstream/main
git push origin main
```

9.3 Referencias entre issues y commits

Tabla 6

Referencias a Issues y Personas en Mensajes de Commit

Sintaxis	Efecto
`fixes #12` o `closes #12`	Cierra el *issue* 12 al fusionar.
`#12`	Enlaza el commit con el *issue*, sin cerrarlo.
`@usuario`	Menciona y notifica a una persona.

Nota. Adaptado de DevTalles (s. f.).

El archivo `CODEOWNERS` asigna revisores automáticos a los *pull requests*, y un archivo `README.md` en un repositorio con el mismo nombre de la cuenta se muestra como portada del perfil (DevTalles, s. f.).

9.4 Síntesis

- El flujo por ramas de funcionalidad aísla cada cambio y lo somete a revisión mediante un *pull request*.
- El *fork* permite contribuir sin permisos de escritura en el repositorio original.
- El remoto *upstream* mantiene el *fork* sincronizado.

10 Etiquetas (tags)

10.1 Uso de etiquetas

Las etiquetas marcan un commit específico, habitualmente para señalar versiones publicadas. La etiqueta anotada es la recomendada, pues almacena autor, fecha y mensaje.

```
git tag                                  # lista las etiquetas
git tag -a v1.0.0 -m "Versión 1.0.0"     # crea una etiqueta anotada
git tag -a v0.9.0 <id_del_commit>        # etiqueta un commit anterior
git show-ref --tags                      # etiquetas con su commit
git push origin --tags                   # publica las etiquetas
git tag -d v1.0.0                        # elimina una etiqueta local
git push origin :refs/tags/v1.0.0        # elimina la etiqueta en el remoto
```

10.2 Síntesis

- Las etiquetas anotadas señalan versiones y se publican de forma explícita con `push --tags`.
- Eliminar una etiqueta local no la elimina en el remoto.

11 Buenas prácticas y mensajes de commit

11.1 Convención de mensajes

La especificación *Conventional Commits* propone una estructura para los mensajes: un tipo, un ámbito opcional entre paréntesis y una descripción breve, seguidos de un cuerpo y de notas al pie opcionales (Conventional Commits, s. f.).

```
tipo(ámbito): descripción breve

cuerpo opcional

nota al pie opcional
```

Tabla 7

Tipos de Commit de Uso Frecuente

Tipo	Uso común	Ejemplo
`feat`	Nueva funcionalidad	`feat(login): agregar validación de campos vacíos`
`fix`	Corrección de errores	`fix(ui): arreglar el centrado del botón de login`
`refactor`	Cambio de código sin alterar la funcionalidad	`refactor(form): simplificar función de inputs`
`style`	Formato y estilo	`style(inputs): aplicar padding uniforme`
`docs`	Documentación	`docs: actualizar README con sección de instalación`
`test`	Pruebas	`test: agregar test para login correcto`
`chore`	Tareas varias (compilación, scripts, dependencias)	`chore: actualizar dependencias del proyecto`
`perf`	Mejoras de rendimiento	`perf(render): optimizar carga del logo`
`build`	Sistema de construcción	`build: configurar PyInstaller para empaquetado`
`ci`	Integración continua	`ci: agregar workflow para GitHub Actions`

Nota. Tipos y ejemplos tomados de las notas del autor (Vega, 2021) y de DevTalles (s. f.).

> PARA AMPLIAR
>
> Según la especificación, `fix` corresponde a una versión de parche y `feat` a una versión menor en el versionado semántico; un cambio incompatible, indicado con `!` antes de los dos puntos o con la nota `BREAKING CHANGE:`, corresponde a una versión mayor (Conventional Commits, s. f.).

11.2 Recomendaciones de uso

- Realizar commits pequeños y frecuentes, con mensajes claros redactados en modo imperativo.
- Trabajar en una rama por funcionalidad o error; no hacerlo directamente en la rama principal.
- Ejecutar `git pull` antes de comenzar para reducir conflictos.
- Revisar `git diff` antes de cada commit.
- Mantener un `README` y un `.gitignore` desde el inicio del proyecto.

11.3 Síntesis

- Un mensaje estructurado facilita la lectura del historial y la automatización de versiones.
- Los commits pequeños, las ramas por tarea y la revisión previa reducen errores y conflictos.

12 Productividad: alias y GitHub CLI

12.1 Alias

Un alias abrevia un comando largo. Pueden definirse en Git o en la terminal:

```
git config --global alias.st status
git config --global alias.lg "log --oneline --graph --all"
git config --global --list               # revisa alias y ajustes globales
alias grafo="git log --all --graph --decorate --oneline"   # alias de la terminal
```

Con la primera línea, `git st` equivale a `git status`; con la segunda, `git lg` muestra el historial gráfico en una línea por commit. Entre los alias más habituales figuran `co` (`checkout`), `br` (`branch`) y `ci` (`commit`) (DevTalles, s. f.).

12.2 GitHub CLI

GitHub CLI (`gh`) permite realizar operaciones de GitHub sin abrir el navegador.

Tabla 8
Comandos de GitHub CLI

Comando	Función
`gh auth login`	Inicia sesión en GitHub desde la terminal.
`gh repo create`	Crea un repositorio.
`gh repo clone usuario/repo`	Clona un repositorio.
`gh pr create`	Abre un *pull request*.
`gh pr list`	Lista los *pull requests* abiertos.
`gh pr checkout 12`	Descarga el *pull request* 12 para probarlo localmente.
`gh issue create`	Crea un *issue*.

Nota. Tomado de DevTalles (s. f.).

12.3 Atajos de teclado en github.com

- `t`: buscador rápido de archivos del repositorio.
- `.`: abre el repositorio en el editor web (github.dev).
- `l`: salta a una línea específica de un archivo.
- `b`: muestra el *blame*.
- `y`: convierte la URL en un enlace permanente al commit.
- `?`: muestra todos los atajos disponibles.

12.4 Síntesis

- Los alias reducen la escritura de comandos largos.
- `gh` integra las operaciones de GitHub en el flujo de la terminal.

13 Markdown para documentación

13.1 Sintaxis básica

Markdown es texto plano con símbolos sencillos que GitHub convierte en formato. Se emplea en el archivo `README.md`, que describe el proyecto y suele ser lo primero que consulta quien visita el repositorio.

Tabla 9
Elementos Básicos de Markdown

Elemento	Sintaxis
Títulos (seis niveles)	`# Título 1` … `###### Título 6`
Negrita, cursiva y tachado	`**negrita**`, `*cursiva*`, `~~tachado~~`
Código en línea	`` `código` ``
Listas	`- elemento` (viñetas), `1. elemento` (numerada), `- [ ] tarea` (casilla)
Enlaces e imágenes	`[texto](https://url)`, `![alt](imagen.png)`
Cita en bloque	`> texto citado`
Línea horizontal	`---`
Emoji, mención y referencia	`:rocket:`, `@usuario`, `#12`

Nota. Adaptado de DevTalles (s. f.).

13.2 Bloques de código, tablas y avisos

Tres comillas invertidas seguidas del nombre del lenguaje activan el resaltado de sintaxis. Las tablas se construyen con barras verticales y una fila de guiones que separa el encabezado; los dos puntos en esa fila (`:---`, `:--:`, `---:`) controlan la alineación. GitHub también admite avisos con las etiquetas `[!NOTE]`, `[!TIP]`, `[!IMPORTANT]`, `[!WARNING]` y `[!CAUTION]` dentro de una cita en bloque (DevTalles, s. f.).

> **CONSEJO**
>
> Debe dejarse una línea en blanco entre párrafos; de lo contrario, se unen en uno solo. Para forzar un salto de línea sin nuevo párrafo se termina la línea con dos espacios. Conviene previsualizar el resultado antes de guardar.

13.3 Síntesis

- Markdown permite documentar proyectos con una sintaxis mínima.
- El `README.md` es la portada del repositorio.

Glosario

Alias. Abreviatura personalizada de un comando.

Área de preparación (*staging area*). Zona donde se agrupan los cambios que formarán parte del próximo commit.

Commit. Instantánea del proyecto guardada en el repositorio.

Conflicto de fusión. Situación en que dos ramas modifican las mismas líneas y Git no puede combinarlas automáticamente.

Fork. Copia de un repositorio ajeno en la propia cuenta de GitHub.

HEAD. Apuntador al commit o rama de trabajo actual.

Origin. Nombre por defecto del remoto principal.

Pull request. Solicitud de fusión de una rama, sujeta a revisión.

Rama (*branch*). Línea de trabajo paralela e independiente.

Remoto. Copia del repositorio alojada en un servidor.

Stash. Almacén temporal de cambios en curso.

Tag (etiqueta). Marca asociada a un commit, usada para señalar versiones.

Referencias

Chacon, S., & Straub, B. (2014). *Pro Git* (2.ª ed.). Apress. https://git-scm.com/book/en/v2

Conventional Commits. (s. f.). *Conventional Commits 1.0.0*. https://www.conventionalcommits.org/en/v1.0.0/

DevTalles. (s. f.). *Git & GitHub cheatsheet* [Hoja de referencia]. https://cursos.devtalles.com

GitHub. (s. f.-a). *Configuring a publishing source for your GitHub Pages site*. GitHub Docs. https://docs.github.com/en/pages/getting-started-with-github-pages/configuring-a-publishing-source-for-your-github-pages-site

GitHub. (s. f.-b). *Generating a new SSH key and adding it to the ssh-agent*. GitHub Docs. https://docs.github.com/en/authentication/connecting-to-github-with-ssh/generating-a-new-ssh-key-and-adding-it-to-the-ssh-agent

Vega, F. (2021). *Curso de Git y GitHub* [Curso en línea]. Platzi.

end?